Gold, Stamps & Phonetech — Sprint 0 & Sprint 1 Technical Spec

Audience: an implementing agent (Codex / ChatGPT) and Regan. **Goal of this spec:** ship the first *usable* version — a self-hosted inventory + photo system — with the seams for everything later already in place.

> Build only what is in this document. Do **not** scaffold AI, valuation, listing generation, or eBay. Those are later sprints. Wherever a future capability needs a "seam," this spec creates the interface and stops.

---

0. Locked decisions this spec implements

- Backend: **Django + Django REST Framework**, **Django ORM**, **PostgreSQL**.
- Frontend: **Vite + React + TypeScript + Tailwind**, PWA-installable.
- **Modular monolith.** Self-hosted; single-user for now (a nullable `owner` FK is the only SaaS seam — not used yet).
- **Composition + strategy**, never ORM inheritance. One `InventoryItem` core record; category-specific data lives in a validated **JSONB** `attributes` field shaped by a **schema registry**.
- **External integrations behind adapter interfaces (ports).** Only `LocalFileStorageAdapter` is implemented. eBay / vision / OCR / metals adapters exist as stubs that raise `NotImplementedError`.
- **eBay is one channel, not the centre.** Zero eBay dependency in Sprint 0/1.
- The app must remain fully valuable with **zero external API access.**

---

1. Repository structure

```
gold-stamps-phonetech/
├── README.md
├── .gitignore
├── docker-compose.yml              # postgres (+ optional adminer)
├── backend/
│   ├── manage.py
│   ├── requirements.txt
│   ├── .env.example
│   ├── pytest.ini
│   ├── config/                     # the Django project
```

```
│   │   ├── __init__.py
│   │   ├── settings/
│   │   │   ├── __init__.py
│   │   │   ├── base.py
│   │   │   ├── dev.py
│   │   │   └── prod.py
│   │   ├── urls.py
│   │   ├── asgi.py
│   │   └── wsgi.py
│   ├── apps/
│   │   ├── __init__.py
│   │   ├── core/                     # shared base models, SKU service, mgmt commands
│   │   │   ├── models.py             # TimeStampedUUIDModel, SkuSequence
│   │   │   ├── services/sku.py
│   │   │   └── management/commands/{seed.py,backup.py,export_items.py}
│   │   ├── catalog/                  # ProductCategory + attribute schema registry
│   │   │   ├── models.py
│   │   │   ├── profiles.py           # schema registry (permissive default in P1)
│   │   │   ├── serializers.py
│   │   │   └── views.py
│   │   ├── locations/                # StorageLocation
│   │   ├── acquisitions/             # AcquisitionRecord
│   │   ├── inventory/                # InventoryItem (the spine)
│   │   │   ├── models.py
│   │   │   ├── serializers.py
│   │   │   ├── filters.py
│   │   │   ├── views.py
│   │   │   └── tests/
│   │   ├── photos/                   # PhotoAsset + image pipeline (MediaService)
│   │   │   ├── models.py
│   │   │   ├── services.py           # MediaService.process_upload(...)
│   │   │   ├── serializers.py
│   │   │   ├── views.py
│   │   │   └── tests/
│   │   ├── dashboard/                # summary endpoint only
│   │   └── api/                      # DRF router aggregation
│   │       └── urls.py
│   ├── integrations/                 # ADAPTER PORTS (interfaces)
│   │   ├── __init__.py
│   │   ├── storage.py                # FileStorageAdapter ABC + LocalFileStorageAdapter (IM
│   │   ├── ebay.py                   # EbayBrowse/Inventory/Media ABCs (STUBS)
│   │   ├── vision.py                 # VisionAdapter, OcrAdapter ABCs (STUBS)
│   │   └── metals.py                 # MetalsPriceAdapter ABC (STUB)
│   └── media_files/                  # uploaded files (gitignored): originals/ processed/ t
└── frontend/
    ├── package.json
    ├── vite.config.ts                # dev proxy /api + /media -> :8000
    ├── tsconfig.json
    ├── tailwind.config.js
    ├── postcss.config.js
    ├── index.html
    ├── public/                       # manifest.webmanifest, icons
```

```
└── src/
    ├── main.tsx
    ├── App.tsx
    ├── routes.tsx
    ├── api/                    # client.ts, items.ts, photos.ts, categories.ts, locat
    ├── types/                  # TS interfaces mirroring serializers
    ├── components/             # StatusBadge, ValueBadge, PhotoThumb, PhotoUploader,
    ├── features/
    │   ├── dashboard/
    │   ├── inventory/          # InventoryGrid, ItemCard, ItemDetail, ItemForm
    │   └── capture/            # AddItem screen
    └── styles/index.css
```

> **Why `apps/` + `config/`:** keeps domains as importable Python packages within one deployable app, so adding `research/`, `valuation/`, `listing/`, `ebay/` later is just new packages. **Why `integrations/` is separate from apps:** ports are infrastructure contracts, not domain apps; this is the line that lets eBay churn without touching business logic.

---

2. Django app layout & responsibilities

App	Owns	Sprint
`core`	`TimeStampedUUIDModel`, `SkuSequence`, SKU service, `seed` / `backup` / `export_items` commands	0
`catalog`	`ProductCategory`, attribute **schema registry** (`profiles.py`)	0
`locations`	`StorageLocation` (self-nesting)	0
`acquisitions`	`AcquisitionRecord`	0
`inventory`	`InventoryItem` (spine), filters, list+detail serializers	0 models / 1 API
`photos`	`PhotoAsset`, `MediaService` image pipeline	0 models / 1 pipeline
`dashboard`	read-only `/dashboard/summary/`	1
`api`	aggregates all DRF routers under `/api/`	1
`integrations`	adapter **ports**; only local storage implemented	0

`INSTALLED_APPS` (Sprint 0): `django.contrib.*`, `rest_framework`, `django_filters`, `corsheaders`, then the local apps above.

---

3. Model definitions

All models inherit a UUID + timestamp base. UUID PKs are deliberate: don't leak inventory counts in URLs, and keep the door open for device sync later.

3.1 `core`

```
# apps/core/models.py
import uuid
from django.db import models

class TimeStampedUUIDModel(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta:
        abstract = True

class SkuSequence(models.Model):
    """One incrementing counter per SKU prefix. Locked on increment."""
    prefix = models.CharField(max_length=12, primary_key=True)
    last_value = models.PositiveIntegerField(default=0)


# apps/core/services/sku.py
from django.db import transaction
from apps.core.models import SkuSequence

def generate_sku(prefix: str) -> str:
    prefix = (prefix or "GSP").upper()
    with transaction.atomic():
        seq = SkuSequence.objects.select_for_update().filter(prefix=prefix).first()
        if seq is None:
            seq = SkuSequence.objects.create(prefix=prefix, last_value=0)
            seq = SkuSequence.objects.select_for_update().get(pk=seq.pk)
        seq.last_value += 1
        seq.save(update_fields=["last_value"])
    return f"{prefix}-{seq.last_value:05d}"   # e.g. STM-00042
```

3.2 `catalog`

```
# apps/catalog/models.py
from django.db import models
from apps.core.models import TimeStampedUUIDModel

class ProductCategory(TimeStampedUUIDModel):
    name = models.CharField(max_length=120)
    slug = models.SlugField(max_length=140, unique=True)
    parent = models.ForeignKey("self", null=True, blank=True,
                               on_delete=models.PROTECT, related_name="children")
```

```
    sku_prefix = models.CharField(max_length=12, default="GSP")
    profile_key = models.CharField(max_length=60, blank=True, default="")  # -> schema regis
    description = models.TextField(blank=True, default="")

    class Meta:
        verbose_name_plural = "product categories"
        ordering = ["name"]

    def __str__(self):
        return self.name


# apps/catalog/profiles.py
"""
Attribute schema registry. In Sprint 1 it is intentionally PERMISSIVE.
Phase 2 replaces the default validator with real per-category schemas
(e.g. stamp: country/year/perforation; gold: weight_g/fineness; phone: imei/storage).
This is the seam that makes composition+strategy work without core migrations.
"""
from typing import Protocol

class AttributeSchema(Protocol):
    profile_key: str
    def validate(self, attributes: dict) -> dict: ...  # returns cleaned attrs or raises Val

class PermissiveSchema:
    profile_key = ""
    def validate(self, attributes: dict) -> dict:
        if not isinstance(attributes, dict):
            raise ValueError("attributes must be an object")
        return attributes

_REGISTRY: dict[str, AttributeSchema] = {}

def register(schema: AttributeSchema) -> None:
    _REGISTRY[schema.profile_key] = schema

def get_schema(profile_key: str) -> AttributeSchema:
    return _REGISTRY.get(profile_key or "", PermissiveSchema())
```

3.3 `locations`

```
# apps/locations/models.py
from django.db import models
from apps.core.models import TimeStampedUUIDModel

class StorageLocation(TimeStampedUUIDModel):
    class LocationType(models.TextChoices):
        SHED = "shed", "Shed"; ROOM = "room", "Room"; SHELF = "shelf", "Shelf"
        BIN = "bin", "Bin"; BOX = "box", "Box"; OTHER = "other", "Other"
```

```
    label = models.CharField(max_length=120)
    type = models.CharField(max_length=20, choices=LocationType.choices, default=LocationTyp
    parent = models.ForeignKey("self", null=True, blank=True,
                               on_delete=models.PROTECT, related_name="children")
    notes = models.TextField(blank=True, default="")

    def __str__(self):
        return self.label
```

3.4 `acquisitions`

```
# apps/acquisitions/models.py
from django.db import models
from apps.core.models import TimeStampedUUIDModel

class AcquisitionRecord(TimeStampedUUIDModel):
    source = models.CharField(max_length=200, blank=True, default="")     # who/where
    acquired_on = models.DateField(null=True, blank=True)
    total_cost = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank=True)
    currency = models.CharField(max_length=3, default="AUD")
    travel_notes = models.TextField(blank=True, default="")               # mileage/petrol/e
    notes = models.TextField(blank=True, default="")

    def __str__(self):
        return self.source or f"Acquisition {self.id}"
```

3.5 `inventory` — the spine

```
# apps/inventory/models.py
from django.contrib.postgres.indexes import GinIndex
from django.db import models
from apps.core.models import TimeStampedUUIDModel
from apps.core.services.sku import generate_sku
from apps.catalog.profiles import get_schema

class InventoryItem(TimeStampedUUIDModel):
    class Status(models.TextChoices):
        CAPTURED = "captured", "Captured"
        NEEDS_ID = "needs_identification", "Needs identification"
        NEEDS_CLEANING = "needs_cleaning", "Needs cleaning"
        NEEDS_RESEARCH = "needs_research", "Needs research"
        READY_TO_LIST = "ready_to_list", "Ready to list"
        LISTED = "listed", "Listed"
        SOLD = "sold", "Sold"
        STORED = "stored", "Stored"
        ARCHIVED = "archived", "Archived"
        IN_BULK_LOT = "in_bulk_lot", "Part of bulk lot"

    class Condition(models.TextChoices):
        NEW = "new", "New"
```

```
    LIKE_NEW = "like_new", "Like new"
    VERY_GOOD = "very_good", "Very good"
    GOOD = "good", "Good"
    ACCEPTABLE = "acceptable", "Acceptable"
    FOR_PARTS = "for_parts", "For parts / not working"
    UNGRADED = "ungraded", "Ungraded / unknown"

sku = models.CharField(max_length=40, unique=True, editable=False, db_index=True)
title = models.CharField(max_length=200, blank=True, default="")
category = models.ForeignKey("catalog.ProductCategory", null=True, blank=True,
                             on_delete=models.PROTECT, related_name="items")
status = models.CharField(max_length=30, choices=Status.choices, default=Status.CAPTURED
condition = models.CharField(max_length=20, choices=Condition.choices, default=Condition
location = models.ForeignKey("locations.StorageLocation", null=True, blank=True,
                             on_delete=models.SET_NULL, related_name="items")
acquisition = models.ForeignKey("acquisitions.AcquisitionRecord", null=True, blank=True,
                                on_delete=models.SET_NULL, related_name="items")

acquisition_cost = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank
estimated_value  = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank
min_price        = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank
target_price     = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank
currency = models.CharField(max_length=3, default="AUD")

notes = models.TextField(blank=True, default="")
attributes = models.JSONField(default=dict, blank=True)  # category-specific, validated

# SaaS seam only — NOT used in Sprint 0/1. No per-user filtering yet.
owner = models.ForeignKey("auth.User", null=True, blank=True,
                          on_delete=models.SET_NULL, related_name="items")

class Meta:
    ordering = ["-created_at"]
    indexes = [
        models.Index(fields=["status"]),
        models.Index(fields=["category"]),
        GinIndex(fields=["attributes"], name="item_attributes_gin"),
    ]

def clean(self):
    profile_key = self.category.profile_key if self.category else ""
    self.attributes = get_schema(profile_key).validate(self.attributes or {})

def save(self, *args, **kwargs):
    if not self.sku:
        prefix = (self.category.sku_prefix if self.category and self.category.sku_prefix
        self.sku = generate_sku(prefix)
    super().save(*args, **kwargs)

@property
def main_photo(self):
    return self.photos.filter(is_main=True).first() or self.photos.order_by("order_index
```

```
    def __str__(self):
        return f"{self.sku} — {self.title or 'Untitled'}"
```

3.6 `photos` — PhotoAsset

```
# apps/photos/models.py
from django.db import models
from apps.core.models import TimeStampedUUIDModel

class PhotoAsset(TimeStampedUUIDModel):
    class Role(models.TextChoices):
        MAIN = "main", "Main"; FRONT = "front", "Front"; BACK = "back", "Back"
        DETAIL = "detail", "Detail / close-up"; BEFORE = "before", "Before"
        AFTER = "after", "After"; OTHER = "other", "Other"
        # stamp/coin-specific roles (perforation, watermark, edge) added in a later sprint

    item = models.ForeignKey("inventory.InventoryItem", on_delete=models.CASCADE, related_na
    role = models.CharField(max_length=20, choices=Role.choices, default=Role.OTHER)
    is_main = models.BooleanField(default=False)
    order_index = models.PositiveIntegerField(default=0)

    original_path  = models.CharField(max_length=500)               # full-res, EXIF-strippe
    processed_path = models.CharField(max_length=500, blank=True, default="")
    thumb_path     = models.CharField(max_length=500, blank=True, default="")

    width  = models.PositiveIntegerField(null=True, blank=True)
    height = models.PositiveIntegerField(null=True, blank=True)
    bytes_original = models.PositiveIntegerField(null=True, blank=True)
    exif_stripped  = models.BooleanField(default=False)
    quality_score  = models.FloatField(null=True, blank=True)        # computed in a later s

    class Meta:
        ordering = ["item", "order_index", "created_at"]
        constraints = [
            models.UniqueConstraint(fields=["item"], condition=models.Q(is_main=True),
                                    name="one_main_photo_per_item"),
        ]
```

3.7 Image pipeline contract (`photos/services.py`)

`MediaService.process_upload(item, django_file, role) -> PhotoAsset` . **Steps, in order:**

1. Open with Pillow; `ImageOps.exif_transpose` to bake in orientation, then convert to `RGB` . (This applies rotation *before* stripping metadata so photos aren't sideways.)
2. **Original** = the transposed, metadata-stripped image at full resolution → save as JPEG q≈92 → `original_path` . *Original pixels preserved; GPS/EXIF gone.* Set `exif_stripped=True` , record `width/height/bytes_original` .

3. **Processed** = longest side ≤ 2000px, JPEG q≈85 → `processed_path`.
4. **Thumbnail** = longest side ≤ 400px, JPEG q≈80 → `thumb_path`.
5. **All writes go through the injected `FileStorageAdapter` (keys like `originals/{item_id}/{uuid}.jpg`). Never write files directly from views.**
6. **If the item has no main photo yet, set this one `is_main=True`; assign `order_index = max(existing)+1`.**

3.8 Adapter ports (`integrations/`)

```
# integrations/storage.py
from abc import ABC, abstractmethod
from pathlib import Path
from django.conf import settings

class FileStorageAdapter(ABC):
    @abstractmethod
    def save(self, key: str, data: bytes, content_type: str = "image/jpeg") -> str: ...
    @abstractmethod
    def open(self, key: str) -> bytes: ...
    @abstractmethod
    def delete(self, key: str) -> None: ...
    @abstractmethod
    def url(self, key: str) -> str: ...

class LocalFileStorageAdapter(FileStorageAdapter):
    def __init__(self, root: Path | None = None, base_url: str = "/media/"):
        self.root = Path(root or settings.MEDIA_ROOT)
        self.base_url = base_url
    def save(self, key, data, content_type="image/jpeg"):
        path = self.root / key
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(data)
        return key
    def open(self, key):  return (self.root / key).read_bytes()
    def delete(self, key):
        p = self.root / key
        if p.exists(): p.unlink()
    def url(self, key):   return f"{self.base_url}{key}"


# integrations/ebay.py  /  vision.py  /  metals.py  — STUBS ONLY
from abc import ABC, abstractmethod

class EbayBrowseAdapter(ABC):
    @abstractmethod
    def search_active(self, query: str, **kw): ...      # Phase 2 (if production access gran

class EbayInventoryAdapter(ABC):
    @abstractmethod
```

```
    def create_or_replace_item(self, sku, payload): ...  # Phase 4
    @abstractmethod
    def create_offer(self, payload): ...                 # Phase 4 — stays UNPUBLISHED
    @abstractmethod
    def publish_offer(self, offer_id): ...               # Phase 4 — human-gated only

class OcrAdapter(ABC):
    @abstractmethod
    def read_text(self, image_bytes): ...                # Phase 5

class VisionAdapter(ABC):
    @abstractmethod
    def classify(self, image_bytes): ...                 # Phase 5

class MetalsPriceAdapter(ABC):
    @abstractmethod
    def spot_price(self, metal: str, currency: str = "AUD"): ...  # Phase 2 (gold)
```

Every stub method body: `raise NotImplementedError("Implemented in Phase X")` **. Do not implement these in Sprint 0/1.**

---

4. Serializers (DRF)

Serializers live in each app; the `api` **app only aggregates routers.**

```
# apps/inventory/serializers.py (shape, not full)
class InventoryItemListSerializer(ModelSerializer):
    category_name = CharField(source="category.name", read_only=True)
    main_thumb_url = SerializerMethodField()
    class Meta:
        model = InventoryItem
        fields = ["id","sku","title","status","condition","category","category_name",
                  "estimated_value","currency","main_thumb_url","created_at"]
    def get_main_thumb_url(self, obj):
        mp = obj.main_photo
        return storage.url(mp.thumb_path) if mp and mp.thumb_path else None

class InventoryItemDetailSerializer(ModelSerializer):
    photos = PhotoAssetSerializer(many=True, read_only=True)
    class Meta:
        model = InventoryItem
        fields = "__all__"            # excludes nothing except auto sku is read-only
        read_only_fields = ["sku","created_at","updated_at"]
    def validate(self, data):
        # run model.clean() schema-registry validation on attributes
        ...
```

Other serializers (straightforward `ModelSerializer` **s):** `ProductCategorySerializer` **,** `StorageLocationSerializer` **(flat:** `parent` **as id),** `AcquisitionRecordSerializer` **,**

`PhotoAssetSerializer` (read returns `original_url/processed_url/thumb_url` via `storage.url(...)`; **upload is not done through this serializer** — see endpoints).

DRF defaults (`config/settings/base.py`): `PageNumberPagination` (page size 50), `SearchFilter` + `DjangoFilterBackend` + `OrderingFilter`, `SessionAuthentication`, `IsAuthenticated`.

---

5. API endpoints (Sprint 1)

All under `/api/`. Auth: session (you log in once at `/admin/`; the Vite dev proxy keeps it same-origin so the cookie + CSRF just work).

Method	Path	Purpose
GET	`/api/items/`	List items. Filters: `status`, `category`, `condition`, `location`, `has_photos`, `min_value`, `max_value`. Search: `?search=` over `title,sku,notes`. Order: `created_at,estimated_value,title`. Paginated. Uses **List** serializer.
POST	`/api/items/`	Create item (Detail serializer). SKU auto-generated.
GET	`/api/items/{id}/`	Item detail incl. nested photos.
PATCH	`/api/items/{id}/`	Partial update (status, fields, attributes).
DELETE	`/api/items/{id}/`	Delete item (cascades photos + files).
POST	`/api/items/{id}/photos/`	**Multipart** upload. Field `image` (required), `role` (optional). Runs pipeline. Returns PhotoAsset.
POST	`/api/items/{id}/photos/reorder/`	Body `{ "order": ["photoId", ...] }`.
GET/PATCH/DELETE	`/api/photos/{id}/`	Edit `role` / `is_main`; delete removes files via adapter. Setting `is_main=true` clears the previous main (respect the unique constraint).
GET/POST/PATCH/DELETE	`/api/categories/`, `/api/categories/{id}/`	Category CRUD.
GET/POST/PATCH/DELETE	`/api/locations/`, `/api/locations/{id}/`	Location CRUD.

GET/POST/PATCH/DELETE	`/api/acquisitions/` , `/api/acquisitions/{id}/`	Acquisition CRUD.
GET	`/api/dashboard/summary/`	`{ total_items, total_estimated_value, currency, by_status: {…}, missing_photos, high_value_unlisted }` .
GET	`/api/items/export.csv`	CSV of items (key fields + photo count). No-lock-in export.

`high_value_unlisted` = count where `estimated_value >= settings.HIGH_VALUE_THRESHOLD` (default 100) and `status` not in (`listed` , `sold` , `archived`). `missing_photos` = items with zero photos.

---

6. Frontend screen / component structure

Stack: React + TS + Tailwind + **react-router-dom** + **@tanstack/react-query** (server state) + **vite-plugin-pwa**. No Redux. Dark mode is the default theme (per the product vision).

Routes (`src/routes.tsx`):

- `/` → Dashboard
- `/inventory` → InventoryGrid
- `/inventory/:id` → ItemDetail
- `/add` → AddItem (capture screen)

Screens / key components:

- `AppShell` — sidebar/bottom nav (Dashboard · Inventory · + Add), dark theme, responsive (sidebar on desktop, bottom bar on phone).
- **Dashboard** — summary cards (Total inventory value, Total items, Needs research, Ready to list, Missing photos) from `/dashboard/summary/` ; each card links into a filtered grid.
- **InventoryGrid** — responsive card grid. `ItemCard` = main thumbnail + title + SKU + `StatusBadge` + `ValueBadge` . Sticky top bar: search box + filter chips (status, category, condition). Pagination or "load more". `EmptyState` when no items.
- **ItemDetail** — `PhotoGallery` (main image + thumbnail strip), all fields, inline edit via `ItemForm` , status changer, and photo management (upload more, set main, reorder via drag or up/down, delete with `ConfirmDialog`).
- **AddItem (capture)** — mobile-first. Photo input uses `<input type="file" accept="image/*" capture="environment" multiple>` to open the phone camera; preview chips before submit. Core fields: title, category (`CategorySelect`), condition, location (`LocationSelect`), acquisition cost,

estimated value, notes. On submit: create item → upload each photo → route to its detail page.

Shared components: `StatusBadge`, `ValueBadge`, `PhotoThumb`, `PhotoUploader`, `PhotoGallery`, `CategorySelect`, `LocationSelect`, `ConfirmDialog`, `EmptyState`.

`src/api/` — `client.ts` (fetch wrapper: base URL, JSON + multipart helpers, reads CSRF token from cookie for unsafe methods), plus one module per resource returning typed data. **`src/types/`** mirrors the serializers (`InventoryItem`, `PhotoAsset`, `ProductCategory`, `StorageLocation`, `DashboardSummary`).

PWA: `manifest.webmanifest` (name, icons, `display: standalone`, dark theme color) + a minimal service worker caching the app shell. **No offline capture queue this sprint** — that's a later feature; note the seam but don't build it.

---

7. Install & run

Prereqs: Python 3.12, Node 20+, Docker (for Postgres) or a local Postgres 16.

Backend

```
cd backend
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env                                   # set SECRET_KEY, DATABASE_URL, DEBUG
docker compose up -d db                                # from repo root, starts Postgres
python manage.py migrate
python manage.py createsuperuser
python manage.py seed                                  # categories, locations, sample items
python manage.py runserver                             # http://localhost:8000  (admin at /a
```

Frontend

```
cd frontend
npm install
npm run dev                                            # http://localhost:5173
```

Log in once at `http://localhost:8000/admin/`, then use the app at `:5173` (the Vite proxy forwards `/api` and `/media` to `:8000`, so the session cookie carries over).

`requirements.txt` (pin via `pip freeze` after install):

```
Django>=5.1,<6.0
djangorestframework>=3.15
django-filter>=24.0
django-cors-headers>=4.4
psycopg[binary]>=3.2
Pillow>=10.4
dj-database-url>=2.2
```

```
python-dotenv>=1.0
pytest>=8.0
pytest-django>=4.8
```

`vite.config.ts` proxy (the part that matters):

```
server: {
  proxy: {
    "/api":   { target: "http://localhost:8000", changeOrigin: true },
    "/media": { target: "http://localhost:8000", changeOrigin: true },
  },
},
```

`docker-compose.yml`

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: gsp
      POSTGRES_USER: gsp
      POSTGRES_PASSWORD: gsp
    ports: ["5432:5432"]
    volumes: ["gsp_pgdata:/var/lib/postgresql/data"]
volumes: { gsp_pgdata: {} }
```

`.gitignore` must include: `.venv/`, `backend/media_files/`, `*.sqlite3`, `.env`, `node_modules/`, `frontend/dist/`, `backups/`.

---

8. Acceptance criteria

Sprint 0 (skeleton + data model) — done when:

1. `python manage.py migrate` runs clean; all models + `SkuSequence` created.
2. Django admin registers and can create/edit `InventoryItem`, `ProductCategory`, `StorageLocation`, `AcquisitionRecord`, `PhotoAsset`. Creating an item with no SKU auto-generates one.
3. `integrations/` ports exist; `LocalFileStorageAdapter` works; all eBay/vision/metals adapter methods raise `NotImplementedError`.
4. Settings split into base/dev/prod; `.env` loaded; CORS + Vite proxy configured.
5. One documented command sequence brings the backend up from a clean clone.

Sprint 1 (usable app) — done when:

1. From the React **Add** screen you can create an item with ≥1 photo and land on its detail page.

2. Photo upload **strips EXIF** (no GPS in output), preserves a full-res original, and produces a processed image and a thumbnail; dimensions and original byte size are recorded; orientation is correct (no sideways photos).
3. SKUs are unique, human-readable, sequential per prefix, and survive rapid sequential creates (transaction-safe; the prefix PK prevents duplicates).
4. The grid lists items with main thumbnail + status badge + value; **search** (title/sku/notes) and **filters** (status/category/condition) work; results paginate.
5. Item detail shows all fields + gallery; you can edit fields, change status, set the main photo, reorder photos, and delete photos.
6. Dashboard shows total item count, total estimated value, counts by key status, and missing-photo count — numbers match a seeded dataset.
7. `manage.py backup` produces a bundle that **restores cleanly on an empty database**, including image files. CSV export downloads.
8. Backend test suite passes (section 9).

---

9. Test plan

Backend (`pytest` + `pytest-django`):

- *Models:* SKU format + uniqueness; `SkuSequence` increments correctly across many creates; `PhotoAsset` one-main-per-item constraint rejects a second main; permissive attribute schema passes a dict and rejects a non-dict.
- *Image pipeline:* feed a fixture JPEG carrying fake EXIF/GPS → assert output has **no GPS tags**, processed ≤ 2000px, thumb ≤ 400px, all three files written through the adapter, `exif_stripped=True`, orientation applied.
- *API (DRF `APIClient`):* item create / list / retrieve / patch / delete; filter by status & category; search by sku; multipart photo upload happy path + rejects a non-image; reorder; set-main clears previous main; `dashboard/summary` totals correct for a seeded set; unauthenticated request is rejected (401/403).
- *Backup:* run `backup`, drop+recreate DB, run restore, assert item + photo rows and files are present.

Frontend (Vitest + React Testing Library — light):

- `ItemCard` renders title/sku/badge; `StatusBadge` maps each status to a colour; `AddItem` blocks submit with no photo / no title; `api/client` builds correct requests (mock `fetch`).
- Playwright E2E is **optional/deferred** — note it, don't build it this sprint.

Manual QA checklist (one pass on a phone): install the PWA, capture an item with 3 photos via the camera input, set a main photo, confirm thumbnails render, filter the grid, run a backup and a

restore.

---

10. Seed data (`apps/core/management/commands/seed.py`)

Idempotent (`get_or_create`). Creates:

- **Categories** (with `sku_prefix`): Stamps `STM` , Coins `COIN` , Phones & Electronics `PH` , Gold `GOLD` , Tools `TOOL` , Collectibles `COL` , Bulk Lots `LOT` , Unknown `UNK` .
- **Locations:** one Shed → two Shelves → a couple of Bins (demonstrates nesting).
- **Sample items:** ~5 across different categories and statuses (e.g. a `captured` stamp lot, a `needs_research` coin, a `ready_to_list` phone) with placeholder `attributes` and estimated values, so the grid and dashboard have content. Photos optional — if a sample image ships in the repo, attach one to one item to exercise the pipeline.

---

11. Backup / export plan

The rule: back up the database **and** the image files, and a restore must be tested (it's an acceptance criterion). Backing up Postgres alone silently loses every photo.

- **`manage.py backup`** → writes `backups/backup-YYYYMMDD-HHMMSS.zip` containing:
  - `db.dump` — `pg_dump` (robust, full fidelity). *Portable fallback:* `dumpdata` JSON for the app models if `pg_dump` isn't available.
  - `media/` — a copy of `MEDIA_ROOT` (originals, processed, thumbs).
  - `manifest.json` — timestamp, row counts, app versions.
- **Restore (documented in README):** restore the DB (`pg_restore` or `loaddata`) → extract `media/` into `MEDIA_ROOT` .
- **`manage.py export_items --csv`** and `GET /api/items/export.csv` → spreadsheet-friendly export (no lock-in).
- Note for later: copy `backups/` off the machine (external drive / cloud bucket); a `BackupService` behind a future `FileStorageAdapter` can automate off-site copies without code changes here.

Hand-off notes for the implementing agent

- Implement **Sprint 0 fully and get migrations + admin green before any frontend work.** The admin is your first usable inventory UI — enter a few real items in it to validate the model before React exists.
- Do **not** add: authentication beyond Django's built-in + session, Celery/Redis, any AI/OCR/vision

call, any eBay call, real per-category attribute schemas, or an offline sync queue. Every one of those has a seam already; filling them is a later sprint.

- **Keep all file I/O behind `FileStorageAdapter`. Keep all future-integration code behind the `integrations/` ports. If you find yourself importing a vendor SDK into an app, stop — it belongs behind a port.**
- **When something is ambiguous, prefer the smaller implementation that satisfies the acceptance criteria.**